

1. Introduction

1.1 Contexte

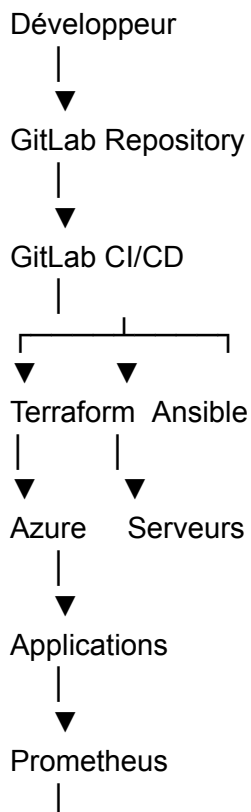
Dans le cadre du projet de modernisation du système d'information de CYNA, la mise en œuvre d'une démarche DevOps vise à automatiser le déploiement, la supervision et la maintenance des infrastructures nécessaires au fonctionnement des services de l'entreprise.

L'objectif est d'améliorer la fiabilité, la disponibilité et la rapidité de mise en production tout en réduisant les interventions manuelles et les risques d'erreurs humaines.

Cette démarche s'inscrit dans la stratégie globale de transformation numérique de CYNA et doit accompagner la croissance future de l'entreprise ainsi que le développement de ses services SaaS et de cybersécurité.

2. Architecture DevOps cible

L'architecture retenue repose sur une chaîne d'automatisation complète permettant de gérer le cycle de vie des infrastructures et des applications.





Cette architecture permet d'assurer une automatisation cohérente depuis le développement jusqu'à l'exploitation.

3. Infrastructure as Code

3.1 Objectifs

L'Infrastructure as Code (IaC) permet de décrire l'infrastructure sous forme de fichiers versionnés.

Cette approche apporte :

- reproductibilité ;
 - traçabilité ;
 - rapidité de déploiement ;
 - réduction des erreurs humaines.
-

3.2 Choix de Terraform

Terraform a été retenu pour le provisionnement des ressources Azure.

Ressources concernées :

- Resource Groups
- Réseaux virtuels
- Sous-réseaux
- Machines virtuelles
- Groupes de sécurité
- Adresses IP publiques

Avantages

- déploiement reproductible ;
- gestion d'état ;
- compatibilité Azure ;
- intégration GitLab CI/CD.

4. Configuration automatisée

4.1 Choix d'Ansible

Ansible est utilisé afin d'automatiser la configuration des systèmes après leur création par Terraform.

Le rôle d'Ansible est complémentaire à celui de Terraform :

Terraform	Ansible
Crée les ressources	Configure les ressources
Provisionne Azure	Installe les services
Réseau	Configuration système
VM	Applications

4.2 Playbooks développés

Les playbooks suivants sont prévus :

hardening_base.yml

Configuration de sécurité des serveurs :

- mises à jour ;
- Fail2ban ;
- pare-feu ;
- SSH sécurisé.

install_monitoring.yml

Déploiement :

- Prometheus ;
- Grafana.

install_graylog.yml

Déploiement :

- MongoDB ;
- OpenSearch ;
- Graylog.

install_gitlab_runner.yml

Installation des GitLab Runners.

backup_config.yml

Sauvegarde automatique des configurations critiques.

5. Gestion des versions et CI/CD

5.1 Git

L'ensemble du code d'infrastructure est stocké dans un dépôt Git unique.

Structure :

infra/
terraform/
ansible/
monitoring/
scripts/
docs/
pipelines/

5.2 GitLab CI/CD

GitLab CI/CD automatise :

- validation Terraform ;
- tests ;
- déploiement ;
- contrôle qualité.

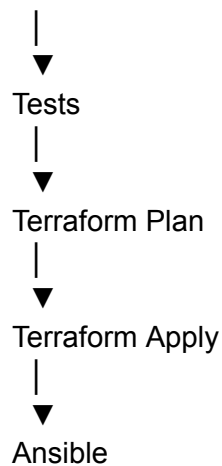
Pipeline :

Commit

|



Validation



6. Monitoring et supervision

6.1 Prometheus

Prometheus collecte les métriques :

- CPU ;
 - mémoire ;
 - disque ;
 - disponibilité ;
 - services.
-

6.2 Grafana

Grafana fournit :

- dashboards ;
- graphiques ;
- alertes.

Alertes prévues

Élément	Seuil
CPU	> 80 %
RAM	> 85 %
Disque	> 90 %

7. Gestion centralisée des logs

7.1 Besoin

L'infrastructure génère quotidiennement de nombreux événements :

- authentications ;
- erreurs applicatives ;
- logs systèmes ;
- événements de sécurité.

Une centralisation est indispensable.

7.2 Choix de Graylog

Graylog a été retenu afin de :

- centraliser les logs ;
- faciliter les investigations ;
- améliorer la supervision.

Sources :

- Linux ;
 - Windows ;
 - Firewall ;
 - Azure ;
 - Applications SaaS.
-

8. PRA / PCA

L'automatisation DevOps participe directement à la stratégie de continuité d'activité.

Les mécanismes prévus sont :

- sauvegarde automatique ;
- reconstruction de l'infrastructure via Terraform ;

- réinstallation automatique des services via Ansible ;
- restauration rapide des configurations.

Cette approche réduit significativement le temps nécessaire à la remise en service d'une plateforme après incident.

9. Livrables DevOps

Les livrables produits seront :

- dépôt Git structuré ;
 - scripts Terraform ;
 - playbooks Ansible ;
 - pipelines GitLab CI/CD ;
 - documentation technique ;
 - dashboards Grafana ;
 - plateforme Graylog ;
 - procédures PRA/PCA ;
 - schémas d'architecture.
-

10. Conclusion

La mise en œuvre de cette architecture DevOps permettra à CYNA de disposer d'un environnement moderne, automatisé et facilement maintenable. L'association de Terraform, Ansible, GitLab CI/CD, Prometheus, Grafana et Graylog garantit une meilleure maîtrise des infrastructures, une réduction des risques opérationnels et une amélioration significative de la disponibilité des services.